

Εθνικό Μετσόβιο Πολυτεχνείο



Εργαστήριο Μικροϋπολογιστών

5η Εργαστηριακή Άσκηση

Χριστόφορος Χαραλάμπους 03121614

Φίλιππος Γιαννακόπουλος 03121629

Ομάδα 17

Ζήτημα 5.1

Το Ζήτημα 5.1 βρίσκεται στο αρχείο erg5.1.c.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define PCA9555_0_ADDRESS 0x40 // A0=A1=A2=0 by hardware
#define TWI_READ 1 // reading from twi device
#define TWI_WRITE 0 // writing to twi device
#define SCL_CLOCK 100000L // twi clock in Hz

//Fsc1=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START 0x08
#define TW_REP_START 0x10
//----- Master Transmitter -----
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28
//----- Master Receiver -----
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b11111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

// Initializes TWI clock
void twi_init() {
    TWSR0 = 0; // Prescaler = 1
    TWBR0 = TWBR0_VALUE; // SCL clock 100kHz
}

// Read one byte from twi device (while requesting more)
unsigned char twi_readAck() {
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDR0;
}

// Read one byte from twi device (followed by a stop condition)
unsigned char twi_readNak() {
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR0 & (1<<TWINT)));
```

```

    return TWDRO;
}

// Sends start condition, address and transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_start(unsigned char address) {
    uint8_t twi_status;

    // Send START
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        return 1;

    // Send device address
    TWDRO = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received.
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK))
        return 1;

    return 0;
}

// Sends start condition, address and transfer direction
// Waits until device is ready
void twi_start_wait(unsigned char address) {
    uint8_t twi_status;

    while (1) {
        // Send START
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // Wait till START transmission is completed
        while (!(TWCR0 & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;
        if ((twi_status != TW_START) && (twi_status != TW_REP_START))
            continue;

        // Send device address
        TWDRO = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);

        // Wait till address transmission is completed and ACK/NACK has been received
        while (!(TWCR0 & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;

```

```

        if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK)) {
            // Device is busy, send STOP condition
            TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

            // Wait till STOP condition is executed and bus released
            while (!(TWCR0 & (1<<TWSTO)));

            continue;
        }

        break;
    }
}

// Send one byte to the TWI device
// Returns 0 for write success, 1 for write failure
unsigned char twi_write(unsigned char data) {
    // Send data to the (already addressed) device
    TWDR0 = data;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till transmission finishes
    while (!(TWCR0 & (1<<TWINT)));

    if ((TW_STATUS & 0xF8) != TW_MT_DATA_ACK)
        return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_rep_start(unsigned char address) {
    return twi_start(address);
}

// Stops data transfer and releases TWI bus
void twi_stop() {
    // Send STOP condition
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCR0 & (1<<TWSTO)));
}

// PCA9555 functions

void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value) {
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg) {
    uint8_t ret_val;

    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
    ret_val = twi_readNak();
    twi_stop();
}

```

```

    return ret_val;
}

int main(void) {
    twi_init();

    // Set PORTB to an input
    DDRB = 0x00;
    PORTB = 0xFF;

    // Set EXT_PORT0 as output
    PCA9555_0_write(REG_CONFIGURATION_0, 0x00);

    while (1) {
        unsigned char input = ~PINB;
        unsigned char A = input & 1;
        unsigned char B = (input>>1) & 1;
        unsigned char C = (input>>2) & 1;
        unsigned char D = (input>>3) & 1;
        unsigned char An = (~A) & 1;
        unsigned char Bn = (~B) & 1;
        unsigned char Dn = (~D) & 1;

        unsigned char F0 = ~((An & B & C) | (Bn & D)) & 1;
        unsigned char F1 = (A | B | C) & (B & Dn);
        F0 |= F1<<1;

        PCA9555_0_write(REG_OUTPUT_0, F0);
    }
}

```

Ζήτημα 5.2

Το Ζήτημα 5.2 βρίσκεται στο αρχείο erg5.2.c.

Για να διαβάσουμε από την πρώτη γραμμή του πληκτρολογίου, γράφουμε τις τιμες 1110 στα IO1_0-3, έτσι ώστε να ενεργοποιηθεί μόνο η γραμμή 0.

Αφού διαβάσουμε την είσοδο, αρκεί απλά να κάνουμε Shift Right 4 φορές και πλέον έχουμε την έξοδο που θέλουμε.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define PCA9555_0_ADDRESS 0x40 // A0=A1=A2=0 by hardware
#define TWI_READ 1 // reading from twi device
#define TWI_WRITE 0 // writing to twi device
#define SCL_CLOCK 100000L // twi clock in Hz

//FscL=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START 0x08
#define TW_REP_START 0x10
//----- Master Transmitter -----
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28
//----- Master Receiver -----
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b1111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

// Initializes TWI clock
void twi_init() {
    TWSR0 = 0; // Prescaler = 1
    TWBR0 = TWBR0_VALUE; // SCL clock 100kHz
}

// Read one byte from twi device (while requesting more)
unsigned char twi_readAck() {
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDRO;
}
```

```

// Read one byte from twi device (followed by a stop condition)
unsigned char twi_readNak() {
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDRO;
}

// Sends start condition, address and transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_start(unsigned char address) {
    uint8_t twi_status;

    // Send START
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        return 1;

    // Send device address
    TWDRO = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received.
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK))
        return 1;

    return 0;
}

// Sends start condition, address and transfer direction
// Waits until device is ready
void twi_start_wait(unsigned char address) {
    uint8_t twi_status;

    while (1) {
        // Send START
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // Wait till START transmission is completed
        while (!(TWCR0 & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;
        if ((twi_status != TW_START) && (twi_status != TW_REP_START))
            continue;

        // Send device address
        TWDRO = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);
    }
}

```

```

// Wait till address transmission is completed and ACK/NACK has been received
while (!(TWCRO & (1<<TWINT)));

// Check TWI Status
twi_status = TW_STATUS & 0xF8;
if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK)) {
    // Device is busy, send STOP condition
    TWCRO = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCRO & (1<<TWSTO)));

    continue;
}

break;
}

// Send one byte to the TWI device
// Returns 0 for write success, 1 for write failure
unsigned char twi_write(unsigned char data) {
    // Send data to the (already addressed) device
    TWDR0 = data;
    TWCRO = (1<<TWINT) | (1<<TWEN);

    // Wait till transmission finishes
    while (!(TWCRO & (1<<TWINT)));

    if ((TW_STATUS & 0xF8) != TW_MT_DATA_ACK)
        return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_rep_start(unsigned char address) {
    return twi_start(address);
}

// Stops data transfer and releases TWI bus
void twi_stop() {
    // Send STOP condition
    TWCRO = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCRO & (1<<TWSTO)));
}

// PCA9555 functions

void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value) {
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg) {
    uint8_t ret_val;

```



```

twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
twi_write(reg);
twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
ret_val = twi_readNak();
twi_stop();

return ret_val;
}

int main(void) {
    twi_init();

    // Set 4 LSB of EXT_PORT0 as output
    PCA9555_0_write(REG_CONFIGURATION_0, 0b00000000);

    // Configure EXT_PORT1
    PCA9555_0_write(REG_CONFIGURATION_1, 0b11110000);

    PCA9555_0_write(REG_OUTPUT_1, 0b00001110);

    while (1) {
        uint8_t input = PCA9555_0_read(REG_INPUT_1);
        input = input>>4;
        input = ~input;
        PCA9555_0_write(REG_OUTPUT_0, input);
    }
}

```

Ζήτημα 5.3

Το Ζήτημα 5.3 βρίσκεται στο αρχείο erg5.3.c.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define PCA9555_0_ADDRESS 0x40 // A0=A1=A2=0 by hardware
#define TWI_READ 1 // reading from twi device
#define TWI_WRITE 0 // writing to twi device
#define SCL_CLOCK 100000L // twi clock in Hz

//FscL=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START 0x08
#define TW_REP_START 0x10
//----- Master Transmitter -----
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28
//----- Master Receiver -----
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b11111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

uint8_t PREVIOUS = 0;

// Initializes TWI clock
void twi_init() {
    TWSR0 = 0; // Prescaler = 1
    TWBR0 = TWBR0_VALUE; // SCL clock 100kHz
}

// Read one byte from twi device (while requesting more)
unsigned char twi_readAck() {
    TWSR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while (!(TWSR0 & (1<<TWINT)));

    return TWDR0;
}

// Read one byte from twi device (followed by a stop condition)
unsigned char twi_readNak() {
```

```

    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDRO;
}

// Sends start condition, address and transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_start(unsigned char address) {
    uint8_t twi_status;

    // Send START
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        return 1;

    // Send device address
    TWDRO = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received.
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK))
        return 1;

    return 0;
}

// Sends start condition, address and transfer direction
// Waits until device is ready
void twi_start_wait(unsigned char address) {
    uint8_t twi_status;

    while (1) {
        // Send START
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // Wait till START transmission is completed
        while (!(TWCR0 & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;
        if ((twi_status != TW_START) && (twi_status != TW_REP_START))
            continue;

        // Send device address
        TWDRO = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);

        // Wait till address transmission is completed and ACK/NACK has been received
        while (!(TWCR0 & (1<<TWINT)));
    }
}

```

```

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK)) {
        // Device is busy, send STOP condition
        TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

        // Wait till STOP condition is executed and bus released
        while (!(TWCR0 & (1<<TWSTO)));

        continue;
    }

    break;
}

// Send one byte to the TWI device
// Returns 0 for write success, 1 for write failure
unsigned char twi_write(unsigned char data) {
    // Send data to the (already addressed) device
    TWDR0 = data;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till transmission finishes
    while (!(TWCR0 & (1<<TWINT)));

    if ((TW_STATUS & 0xF8) != TW_MT_DATA_ACK)
        return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_rep_start(unsigned char address) {
    return twi_start(address);
}

// Stops data transfer and releases TWI bus
void twi_stop() {
    // Send STOP condition
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCR0 & (1<<TWSTO)));
}

// PCA9555 functions

void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value) {
    PREVIOUS = value;
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg) {
    uint8_t ret_val;

    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);

```

```

    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
    ret_val = twi_readNak();
    twi_stop();

    return ret_val;
}

// LCD Commands

void write2(unsigned char input) {
    unsigned char prev = PREVIOUS;

    unsigned char write = (input & 0xF0) | (prev & 0x0F);
    PCA9555_0_write(REG_OUTPUT_0, write);

    write |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, write);
    write &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, write);

    write = ((input & 0x0F) << 4) | (prev & 0x0F);
    PCA9555_0_write(REG_OUTPUT_0, write);

    write |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, write);
    write &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, write);
}

void lcd_data(unsigned char input) {
    unsigned char prev = PREVIOUS;
    prev |= (1<<2);

    PCA9555_0_write(REG_OUTPUT_0, prev);
    write2(input);
    _delay_us(250);
}

void lcd_command(unsigned char input) {
    unsigned char prev = PREVIOUS;
    prev &= 0b11111011;

    PCA9555_0_write(REG_OUTPUT_0, prev);
    write2(input);
    _delay_us(250);
}

void lcd_nextline() {
    lcd_command(0b11000000);
}

void lcd_clear() {
    lcd_command(0x01);
    _delay_ms(5);
}

void lcd_init() {
    _delay_ms(200);

    // Switch to 8bit mode
    unsigned char write = 0x30;

```

```

PCA9555_0_write(REG_OUTPUT_0, write);

unsigned char temp = write;

temp |= (1<<3);
PCA9555_0_write(REG_OUTPUT_0, temp);
temp &= 0b11110111;
PCA9555_0_write(REG_OUTPUT_0, temp);
_delay_us(250);

temp = write;

// Switch to 8bit mode
PCA9555_0_write(REG_OUTPUT_0, write);

temp |= (1<<3);
PCA9555_0_write(REG_OUTPUT_0, temp);
temp &= 0b11110111;
PCA9555_0_write(REG_OUTPUT_0, temp);
_delay_us(250);

temp = write;

// Switch to 8bit mode
PCA9555_0_write(REG_OUTPUT_0, write);

temp |= (1<<3);
PCA9555_0_write(REG_OUTPUT_0, temp);
temp &= 0b11110111;
PCA9555_0_write(REG_OUTPUT_0, temp);
_delay_us(250);

// Switch to 8bit mode
write = 0x20;
temp = write;
PCA9555_0_write(REG_OUTPUT_0, write);

temp |= (1<<3);
PCA9555_0_write(REG_OUTPUT_0, temp);
temp &= 0b11110111;
PCA9555_0_write(REG_OUTPUT_0, temp);
_delay_us(250);

lcd_command(0x28);

lcd_command(0x0c);

lcd_clear();

lcd_command(0x06);
}

int main(void) {
    twi_init();

    // Configure EXT_PORT1 as output
    PCA9555_0_write(REG_CONFIGURATION_0, 0x00);

    // Enable LCD
    lcd_init();

```

```

// Display name and surname
char first_name[] = "CHRISTOFOROS";
char last_name[] = "CHARALAMBOUS";

char first_nameF[] = "FILIPPOS";
char last_nameF[] = "GIANNAKOPOULOS";

while (1) {
    for (int i = 0; i < 12; i++) {
        lcd_data(first_name[i]);
    }
    lcd_nextline();
    for (int i = 0; i < 12; i++) {
        lcd_data(last_name[i]);
    }
    _delay_ms(2000);
    lcd_clear();
    for (int i = 0; i < 8; i++) {
        lcd_data(first_nameF[i]);
    }
    lcd_nextline();
    for (int i = 0; i < 14; i++) {
        lcd_data(last_nameF[i]);
    }
    _delay_ms(2000);
    lcd_clear();
}
}

```